E-HEALTH MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

KAVIYA S

(8115U24AD047)

in partial fulfillment of requirements for the award of the course of

CGB1201 –JAVAPROGRAMMING

IN

DEPARTMENT OF

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Under the guidance of

Mrs.G.ISWARYA

Department of Artificial Intelligence and Data Science

K.RAMAKRISHNAN COLLEGE OF ENGINEERING (AUTONOMOUS)
SAMAYAPURAM,TRICHY 621112

ANNA UNIVERSITY
CHENNAI 600 025

DECEMBER2025

E-HEALTH MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

KAVIYA S(8115U24AD047)

in partial fulfillment of requirements for the award of the course of

CGB1201 –JAVA PROGRAMMING

IN

DEPARTMENT OF

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Under the Guidance of

Mrs.G.ISWARYA

Department of Artificial Intelligence and Data Science

K.RAMAKRISHNAN COLLEGE OF ENGINEERING



K.RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
Under
ANNAUNIVERSITY,CHENNAI



K.RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
Under
ANNA UNIVERSITY,CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled **"E-HEALTH MANAGEMENT SYSTEM**"is the bonafide work of **KAVIYA S(8115U24AD047)**,who carried out the project work under my supervision.

SIGNATURE

Dr.B.KIRANBALAM.E.,M.B.A.,Ph.D.,
HEAD OF THE DEPARTMENT
ASSOCIATE PROFESSOR,
Department of Artificial Intelligence
and Data Science,
K.Ramakrishnan College of
Engineering,(Autonomous)
Samayapuram,Trichy.

SIGNATURE

Mrs.G.ISWARYA,M.E.,M.B.A.,
SUPERVISOR
ASSISTANT PROFESSOR,
Department of Artificial Intelligence
and Data Science,
K.Ramakrishnan College of
Engineering,(Autonomous)
Samayapuram,Trichy.

SIGNATURE OF INTERNAL EXAMINER

NAME:
DATE:

SIGNATURE OF EXTERNAL EXAMINER

NAME:
DATE:

K.RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
Under
ANNA UNIVERSITY, CHENNAI

DECLARATION BY THE CANDIDATE

I declare that to the best of my knowledge the work reported here in has been composed solely by me and that ithasnotbeeninwholeorinpartinany previous application for a degree.

Submitted for the project Viva-Voceheld at K.Ramakrishnan College of Engineering on_____.

SIGNATURE OF THE CANDITATE

ACKNOWLEDGEMENT

I thank the almighty GOD, without whom it would not have been possible for us to complete our project. I wish to address our profound gratitude to **Dr.K.RAMAKRISHNAN**, Chairman, K.Ramakrishnan College of Engineering(Autonomous), who encouraged and gave us all help throughout the course.

I extend myheartygratitude and thanks to our honorable and grateful Executive Director **Dr.S.KUPPUSAMY, B.Sc., MBA., Ph.D.,** K.Ramakrishnan College of Engineering(Autonomous).

I glad to thank our Principal **Dr.D.SRINIVASAN, M.E.,Ph.D., FIE.,MIIW.,MISTE., MISAE., C.Engg**, for giving us permission to carry out this project.

I wishto convey my sincere thanks to **Dr.B.KIRAN BALA, M.E., M.B.A., Ph.D.**, Head of the Department, Artificial Intelligence and Data Science for giving us constant encouragement and advice throughout the course.

Iamgratefulto**Mrs.G.ISWARYA,M.E.M.B.A.**,AssistantProfessor,Artificial Intelligenceand Data Science, K. Ramakrishnan College of Engineering (Autonomous), for her guidance and valuable suggestions during the course of study.

Finally,I sincerely acknowledged innoless terms all our staff members,our parents and, friends for their co-operation and help at various stages of this project work.

KAVIYA S (8115U24AD047)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

VISION OF THE INSTITUTION

To achieve aprominent position among the top technical institutions.

MISSION OF THE INSTITUTION

M1:To be stow standard technical education parexcellence through state of the art infrastructure, competent faculty and high ethical standards.

M2:To nurturere search and entrepreneurial skills among students in cutting edge technologies.

M3: To provide education for developing high-quality professionals to transform the society.

VISION OF THE DEPARTMENT

To prove excellence in Data Science research,education and innovation with AItools.

MISSION OF THE DEPARTMENT

M1:Tocontributeforgreatercollaborationwithacademiaand businesses.

M2:Toimpart qualityandresearchbasededucationtopromoteinnovationsprovidingsmartsolutions in multi-disciplinary area of Artificial Intelligence and Data Science.

M3:ToprovideeminentDataScientiststoservehumanity

PROGRAMEDUCATIONALOBJECTIVES(PEOS)

Our graduatesshall

PEO1:TocreateGraduateswithsuccessfulcareerinthefieldofDataScienceinallindustriesor pursue higher education and research or evolve as entrepreneur.

PEO2:ToequiptheGraduateswiththeabilityandattitudetoadapttoemergingtechnological changes in the field of expert systems.

PEO3:Toexcelthestudentsassociallycommittedengineerswithhighethicalvalues,leadership qualities and openness for the needs of society.

PROGRAMOUTCOMES

Engineeringstudentswillbeableto:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conductinvestigationsofcomplexproblems:** Useresearch-basedknowledgeandresearch methodsincludingdesignofexperiments,analysisandinterpretationofdata,andsynthesisof the information to provide valid conclusions.
5. **Moderntoolusage:**Create,select,andapplyappropriatetechniques, resources,andmodern engineeringandITtoolsincludingpredictionandmodelingtocomplexengineeringactivities with an understanding of the limitations.
6. **Theengineerandsociety:**Applyreasoninginformed bythe contextualknowledgetoassess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutionsinsocietalandenvironmentalcontexts,anddemonstratetheknowledgeof,andneed for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individualandteamwork:** Functioneffectivelyasanindividual,andasamemberorleader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineeringcommunityandwithsocietyatlarge,suchas,beingabletocomprehendandwrite effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance:** Demonstrate knowledge and understanding of the engineeringandmanagement principlesandapplythesetoone'sownwork,asamemberand leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAMSPECIFICOUTCOMES(PSOs)

- **PSO1:** To develop optimized Data Science Solutions, through analysis, design, implementation, and evaluation to give technological solutions for real-time societal issues.
- **PSO2:**Toemployadvancedanalyticplatformsincreatinginnovativecareerpathsto become best data scientists.

ABSTRACT

The E-Health Management System is an integrated digital platformdesigned to enhance the efficiency, accessibility, and quality of healthcare services. It leverages information and communication technologies to streamline medical data management, improve patient–doctor interactions, and support real-time decision making. The system enables secure storage and retrieval of electronic health records (EHRs), online appointment scheduling, remote consultations, prescription management, and continuous health monitoring through IoT-enabled devices. By automating routine tasks and reducing manual errors, the system improves operational performance in hospitals and clinics. It also empowerspatientsbyprovidingpersonalizedhealthinformation,reminders,and access to their medical history anytime and anywhere. Overall, the E-Health Management System promotes a more transparent, efficient, and patient-centric healthcare environment, contributing to better diagnostics, reduced treatment delays, and enhanced public health outcomes.

The E-Health Management System is a digital healthcare platform designed to streamline the management of patient information, medical records, appointments, and hospital operations. The system provides an efficient and user-friendly interface for doctors, patients, and administrative staff to interact seamlessly. It enables patients to register, book appointments, access prescriptions, and view medical history online, reducing manual paperwork and waiting time. Doctors can manage patient records, update diagnoses, and monitor treatment progress through an integrated dashboard. The system also supports secure data storage, quick retrieval of patient information, and real-time communication between healthcare providers and patients. By automating key healthcare processes, the E-Health Management System enhances accuracy, improves decision-making, and increases the overall efficiency of healthcare delivery. This project demonstrates the importance of technology in creating accessible, reliable, and modern medical services.

ABSTRACTWITHPOs ANDPSOsMAPPING

CO5:BUILDJAVAAPPLICATIONSFORSOLVINGREAL-TIMEPROBLEMS.

ABSTRACT	POsMAPPED	PSOsMAPPED
	PO4 PO5 PO6 PO7 PO8	**PSO l** **PSO2**

Note:**1**-Low,2-Medium,3-High

TABLEOFCONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	viii
	LISTOFFIGURES	x
	ABBREVIATIONS	xi
1	INTRODUCTION	**1**
	1.1Objective	**1**
	1.2Overview	**1**
	1.3JavaProgrammingconcepts	**2**
2	PROJECTMETHODOLOGY	**5**
	2.1ProposedWork	**5**
	2.2BlockDiagram	**6**
3	MODULEDESCRIPTION	**7**
	3.1 PatientManagementModule	**7**
	3.2AppointmentSchedulingModule	**7**
	3.3 ElectronicMedicalRecords(EMR)Module	**8**
	3.4 PrescriptionManagementModule	8
	3.5 DoctorManagementModule	9
	3.6 RemoteHealthMonitoringModule	9
		9
4	CONCLUSION & FUTURE SCOPE	10
	4.1Conclusion	10
	4.2FutureScope	11
	APPENDIXA(SOURCECODE)	12

LISTOFFIGURES

FIGURENO	FIGURENAME	PAGE
2.2.1	COLLEGEHOSTELBOOKINGSYSTEM BLOCKDIAGRAM	6

LISTOF ABBERIVATIONS

ABBREVATION	EXPLANATION
1. **CMS**	ContentManagementSystem
2. **DBMS**	DatabaseManagementSystem
3. **UI**	UserInterface
4. **UX**	UserExperience
5. **PHP**	HypertextPreprocessor
6. **HTML**	HypertextMarkupLanguage
7. **CSS**	CascadingStyleSheets
8. **JS**	JavaScript
9. **API**	ApplicationProgrammingInterface
10. **SMS**	ShortMessageService

CHAPTER1
INTRODUCTION

10.1 Objective

The primary objective of the E-Health Management System is to transform traditionalhealthcareintoasmart,digital,andhighlyefficientenvironmentby integrating modern technologies for seamless management of patient and hospital information. This system aims to centralize all medical data through secure Electronic Health Records (EHRs), enabling instant access to patient history, diagnostic reports, prescriptions, and treatment plans. It focuses on improving the overall accessibility of healthcare by offering online appointment booking, virtual consultations, automated reminders, and 24/7 access to medical services. Another major objective is to reduce the operational burden on healthcare staff by automating routine tasks such as patient registration, scheduling, billing, report generation, and prescription handling, thereby minimizing human errors and increasing accuracy. The system also strives to enhance patient–doctor communication through real-timemessagingandtelemedicineservices,ensuringcontinuouscareevenfrom remote locations. With strong emphasis on privacy and security, the system implements robust authentication, authorization, and data encryption techniques to safeguard sensitive medical information. It further aims to integrateIoT-basedhealthmonitoringdevicestotrackvitalparametersinreal-time, helping doctors make informed decisions using data analytics and predictive insights.

10.2 Overview

The E-Health Management System is a digital platform that replaces traditional paper-

basedhealthcareprocesseswithsecureelectronicrecords.Itallowspatientstobook

appointments online, view their medical reports, receive digital prescriptions, and communicate with doctors easily. Doctors can quickly access patient history, update treatment details, and provide better and faster medical care throughthe system.

The system also helps hospitals by automating tasks such as registration, scheduling, billing, and report management, reducing workload and errors. With strong security to protect patient data and the ability to support real-time health monitoring, the E-Health Management Systemmakeshealthcaremoreefficient,accurate,andconvenient forboth patients and healthcare providers.

10.3 JavaProgrammingConcepts

The E-Health Management System is built using the core principles ofObject-Oriented Programming (OOP) in Java, which allows differenthealthcare components such as patients, doctors, appointments,prescriptions, and medical

records to be represented as separate classes.Conceptslikeinheritance,polymorphism,encapsulation,andabstraction help create a flexible and reusable system design. For example, bothdoctors and patients can inherit common attributes from a single Userclass,whileencapsulationprotectssensitivepatientdata.Abstraction and interfaces help define common behaviors and ensure that the systemfollows a clean and modular structure.

To manage large amounts of medical information efficiently, the systemuses JDBC (Java Database Connectivity) to connect Javaapplicationswith databases like MySQL for storing patient details, appointmentschedules, and treatment histories. Java's Collections Framework(ArrayList, HashMap, LinkedList) is used to store dynamic lists ofrecords retrieved from the database. Exception handling ensures smoothoperationbycatchingandmanagingruntimeerrorssuchasinvalid logins, missing data, database disconnections, or wrong user input. Fordesigning interactive screens, Java GUI frameworks like Swing orJavaFXareused todevelopuser-friendly interfacesforlogin, registration,appointmentbooking,andviewingmedicalreports.

Advanced Java concepts are also used to make thesystem more efficient andsecure.Multithreadinghelpsrunbackgroundtaskssuchassending reminders, updating records, or handling multiple users at the same timewithout slowing down the system. Java networking concepts support systemsthat operate online by enabling communication between client devices and acentralserver.Securityfeaturessuchaspasswordencryption,input validation, and role-based authentication protect patient privacy and sensitivemedical data. By combining these core and advanced Java concepts, the E-Health Management System becomes a powerful, secure, and scalableapplicationsuitableforreal-worldhealthcareenvironments.

InconclusiontheEHealthManagementSystemprovidesamodernandefficientsolution for managing healthcare services by replacing traditional paper-basedprocesseswithsecureandautomateddigitaloperations.Byintegratingelectronic healthrecords, online appointment booking, doctor–patient communication, and real-time data access, the system greatly improves the speed, accuracy, and accessibility of healthcare delivery. It reduces the workload on hospital staff, minimizes human errors, and ensures better coordination between patients and healthcare providers. With the support of Java programming concepts such as OOP, database connectivity, exception handling, and GUI development, the system becomes robust, user-friendly, and scalable. Overall, the E-Health ManagementSystembringssignificantimprovementstohealthcaremanagement, making

it more organized, reliable, and patient-centric.

CHAPTER2

PROJECTMETHODOLOGY

2.1 ProposedWork

The proposed work focuses on developing a fully digital and user-friendly E-Health Management System that simplifies the management of healthcare services for patients, doctors, and hospital administrators. The system aims to provideacentralizedplatformwherepatientrecords,appointments,prescriptions,

andcommunicationcanbehandledefficiently. Theproposedsystemwillreplace manual paperwork with a secure database that stores all medical information electronically, allowing instant access to patient history, reports, and treatment details. Patients will be able to register online, book appointments, receive reminders,andcommunicatewithdoctorsthroughthesystem,reducing hospital visitsandwaitingtime.

For healthcare providers, the proposed work includes building modules for viewing patient details, updating diagnoses, generating prescriptions, and monitoring treatment progress. Admin-level features such as managing doctors, scheduling appointments, and generating reports will also be included. The system will implement Java-based technologies such as OOP concepts, JDBC databaseconnectivity,exceptionhandling, collections,andGUItoolslikeSwing or JavaFX to create an efficient and scalable platform. Security features like authentication, password encryption, and role-based access will be integrated to protectsensitivemedicaldata.Overall,theproposedworkaimstodeliverasmart, secure, and automated healthcare management solution that enhances service quality and improves operational efficiency.

2.2 BlockDiagram

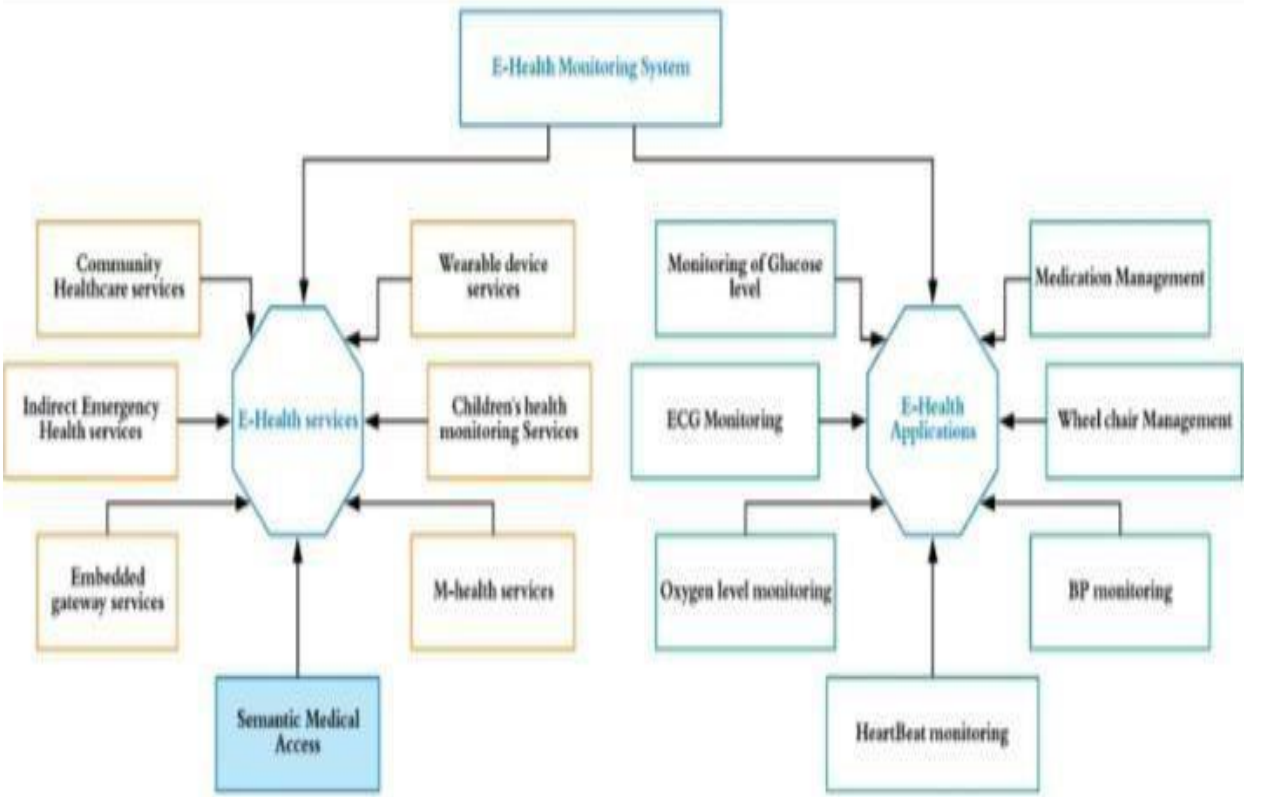


Fig.2.2.1e-healthmanagementsystem

CHAPTER3

MODULE DESCRIPTION

TheE-HealthManagementSystemisdesignedtodigitalizeand streamlinehealthcareactivitiesbyintegratingpatientinformation, hospitaloperations,anddoctor–patientinteractionswithinasingle platform.Thesystemensuresefficienthandlingofmedicaldata, reducesmanualworkload,andimprovesthequalityofhealthcare services.Itincludesmodules forpatient registration,appointments, medical records, prescriptions, billing, andremote monitoring. Each moduleworkscollaborativelythroughacentralizeddatabaseto provide accurate, secure, and real-time healthcare management.

3.1 PatientManagementModule

The Patient Management Module is responsible for handling all essential information related to patients within the healthcare system. It allows new patients toregisterandcreate adigitalprofile thatincludes personaldetails, medicalbackground,contactinformation,andemergencydata.Themodule ensures that patient records are consistently updated and stored securely, allowing healthcare professionals to access accurate information when needed. By maintaining comprehensive patient histories, this module improves the quality of treatment and reduces repetitive data entry during future visits.

3.2 AppointmentSchedulingModule

The Appointment Scheduling Module simplifies the process of booking, rescheduling, and cancelling medical appointments. Patients can view the availabilityofdifferentdoctorsandchooseasuitabletimeslotbasedontheir needs. This module helps hospitals manage patient flow efficiently by reducing overcrowding and long waiting times. It also supports automated reminder notifications via SMSoremail, ensuring that patients do not miss their appointments. The module enhances overall convenience for both healthcare providers and patients.

3.3 ElectronicMedicalRecords(EMR)Module

TheEMRModulestoresandmanagesalldigitalmedicalrecordsofpatients in a secure and organized manner. It includes details such as diagnoses, medications, lab reports, allergies, and treatment histories. Doctors can access these records instantly to make informed clinical decisions. The module ensures data accuracy, confidentiality, and easy retrieval, eliminating the need for physical files and paperwork. By centralizing medicalinformation,itsupportsfasterdiagnosisandimprovesthecontinuity of care.

3.4 PrescriptionManagementModule

The Prescription Management Module enables doctors to create, store, and manageelectronicprescriptionsforpatients.Iteliminateshandwrittenerrors and ensures clarity in dosage instructions, medication types, and treatment duration. Patients can easily access their prescribed medicines in digital format, which enhances convenience and reduces confusion. Additionally, the module maintains a complete record of past prescriptions, helping doctors review medication histories and avoid drug interactions. This leads to safer and more accurate treatments.

3.5 DoctorManagementModule

TheDoctorManagementModulemaintainsdetailedinformationabout

doctors and healthcare staff, such as their qualifications, areas of specialization, contact details, and available consulting hours. It helps administratorsefficientlymanagedoctorschedulesandassignappointments

accordingly. This module ensures that patients are matched with the appropriatedoctorbasedontheirmedicalneeds.Byorganizingandupdating staff information, it enhances coordination between departments and supports smooth hospital operations.

3.6 RemoteHealthMonitoringModule

TheRemoteHealthMonitoringModulesupportscontinuoushealthtracking for patients using wearable devices or self-reported data. It collects vital signs such as heart rate, blood pressure, glucose levels, and oxygen saturation, providing real-time insights into a patient's health condition. Doctors can monitor patients remotely and receive alerts when abnormal readings are detected. This module is especially useful for chronic disease management, elderly care, and telehealth services, allowing timely interventions and improved patient safety.

CHAPTER4

CONCLUSION &FUTURE SCOPE

4.1CONCLUSION

TheCollegeHostelBookingSystemisacomprehensivesolutiondesignedtostreamline andenhancethe hostelmanagement processineducationalinstitutions.Byautomatingcritical functions suchas roombooking, allocation, fee management, and communication, the system addressestheinefficienciesoftraditionalmanualprocesses.Itprovidesauser-friendlyplatform that ensuresconvenience forstudentsandefficiencyforadministrators,contributingtoamore organized and transparent hostel environment. One of the key strengths of this system is its modular design, whichallows eachaspect ofhostelmanagement to be handled independently whilemaintainingseamlessintegrationacrossallcomponents.TheUserManagement Module ensuressecureaccessandrole-basedfunctionalitiesforallusers,whiletheRoomManagement andBookingModulessimplifyroomallocationandavailabilitytracking.TheFeeManagement Module streamlines financial transactions, and the Notification Module enhances communication and keeps users informed. Together, these modules create a robust and cohesivesystem.Thesystemalsoprioritizessecurityandscalability,ensuringthatitcanhandle alargenumberofusersandtransactionswithoutcompromisingdataintegrity.Featuressucha s secure logins, encryption, and audit trails safeguard user information and maintain accountability. Additionally, the system's analytics and reporting capabilities enable administratorstomakeinformeddecisions,optimizeresources,andplanforfutureexpansion s, ensuring long-term benefits for the institution. By incorporating feedback and grievance mechanisms, the system emphasizes user satisfaction and responsiveness. Students can share theirexperiencesandreport issueseasily,whileadministratorscantrackandresolveproblems effectively. This fosters trust and improves the overall quality of hostel services. The system also providesvaluable insightsthroughreportingtools,enablingcontinuousimprovement and better resource management.

4.1 FUTURESCOPE

The**futurescope**oftheCollegeHostelBookingSystemisvast,withopportunities for further enhancement to meet the evolving needs of educational institutions and students. One major area of development is the integration of mobile applications. A mobile version of the system would provide students with greater flexibility, enabling them to book rooms, make payments, and receive notifications directly from their smartphones.

Another promising expansion involves the integration of**AI and machine learning** for smarter room allocation and predictive analytics. By analyzingbookingpatterns,studentpreferences,andhistoricaldata,thesystemcouldautomati callysuggest optimalroomassignments or predict peak booking periods, helping administrators plan better and reduce room shortages.

Lastly, the systemcould be enhanced with **advanced reporting and integration with other campus management systems**. By integrating the hostel booking system with student management systems, academic calendars, or transportation services, institutions can provide a more holistic service to students

APPENDIXA(SOURCECODE)

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.Font;
import java.awt.event.*;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

/**
 * EHealthSystem.java
 *
 * - Single-file Swing app modeled exactly after the
AttendanceSystem style.
 * - Roles: DOCTOR, PATIENT
 * - Auto-generated patient login when adding patient (patientN /
NNN)
 * - Patients can request appointments and chat with doctor
 * - Doctors can accept/reject appointments, add notes, chat with
patient
 *
 * Put this file in src/ and compile:
 * javac -d ../bin EHealthSystem.java
 * Run from bin:
 * java EHealthSystem
 */
public class EHealthSystem {

    // --- Data Structures ---
     static Map<String, Patient> patients = new LinkedHashMap<>();
     static Map<String, Doctor> doctors = new LinkedHashMap<>();
  static Map<String, User> users = new HashMap<>(); // username -
> User
```

```
    static Map<String, Appointment> appointments = new
LinkedHashMap<>(); // apptId -> Appointment

    static DateTimeFormatter DF =
DateTimeFormatter.ofPattern("yyyy-MM-dd");
    static DateTimeFormatter DT =
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");

    public static void main(String[] args) {
        seedData();
        SwingUtilities.invokeLater(LoginFrame::new);
    }

    // --- Seed sample data (auto users) ---
    static void seedData() {
        // Doctors
        doctors.put("D100", new Doctor("D100", "Dr. Meena",
"Cardiology"));
        doctors.put("D101", new Doctor("D101", "Dr. Anand",
"General"));

        // Patients
        patients.put("P200", new Patient("P200", "Kavya"));
        patients.put("P201", new Patient("P201", "Arun"));

        // Users (doctors)
        users.put("doctor1", new User("doctor1", "111",
Role.DOCTOR, "D100"));
        users.put("doctor2", new User("doctor2", "222",
Role.DOCTOR, "D101"));

        // Users (patients) auto style
        users.put("patient1", new User("patient1", "111",
Role.PATIENT, "P200"));
        users.put("patient2", new User("patient2", "222",
Role.PATIENT, "P201"));

        // Example appointment (pending)
        Appointment a = new Appointment(nextAppointmentId(),
```

```
"P200", "D100", LocalDate.now().plusDays(1), "10:00");
        appointments.put(a.id, a);
    }

    // --- Utilities for auto numbering ---
    static int getNextPatientNumber() {
        int max = 0;
        for (User u : users.values()) {
            if (u.role == Role.PATIENT && u.username != null &&
u.username.startsWith("patient")) {
                String suffix = u.username.substring("patient".length());
                try {
                    int n = Integer.parseInt(suffix);
                    if (n > max) max = n;
                } catch (NumberFormatException ignored) {}
            }
        }
        return max + 1;
    }

    static int apptCounter = 500;
    static String nextAppointmentId() { apptCounter++; return "A" +
apptCounter; }

    // --- Domain classes ---
    enum Role { PATIENT, DOCTOR }

    static class User {
        String username, password;
        Role role;
        String personId; // patientId or doctorId

        User(String u, String p, Role r, String pid) { username = u;
password = p; role = r; personId = pid; }
    }

    static class Doctor {
        String id, name, speciality;
```

```
        Doctor(String id, String name, String speciality) { this.id = id;
this.name = name; this.speciality = speciality; }
    }

    static class Patient {
        String id, name;
        // map of date -> appointment present on that date? For
mapping to attendance system style we'll compute accepted %
        Map<LocalDate, String> visits = new TreeMap<>();

        Patient(String id, String name) { this.id = id; this.name = name;
}
    }

    static class Message {
        String sender; Role senderRole; String text; LocalDateTime ts;
        Message(String s, Role r, String t) { sender = s; senderRole = r;
text = t; ts = LocalDateTime.now(); }
        public String toString() { return "[" + ts.format(DT) + "] " +
sender + ": " + text; }
    }

    enum Status { PENDING, ACCEPTED, REJECTED }

    static class Appointment {
        String id;
        String patientId;
        String doctorId;
        LocalDate date;
        String time;
        Status status = Status.PENDING;
        String doctorNotes = "";
        List<Message> messages = new ArrayList<>();

        Appointment(String id, String patientId, String doctorId,
LocalDate date, String time) {
            this.id = id; this.patientId = patientId; this.doctorId =
doctorId; this.date = date; this.time = time;
```

```
    }
}

// ---------- LOGIN FRAME ----------
static class LoginFrame extends JFrame {
    JTextField userField;
    JPasswordField passField;

    LoginFrame() {
        setTitle("E-Health Login");
        setSize(360, 220);
        setLayout(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JLabel title = new JLabel("E-Health System");
        title.setFont(new Font("SansSerif", Font.BOLD, 16));
        title.setBounds(110, 10, 200, 25);
        add(title);

        add(new JLabel("Username:")).setBounds(20, 50, 80, 25);
        userField = new JTextField();
        userField.setBounds(110, 50, 200, 25);
        add(userField);

        add(new JLabel("Password:")).setBounds(20, 90, 80, 25);
        passField = new JPasswordField();
        passField.setBounds(110, 90, 200, 25);
        add(passField);

        JButton loginBtn = new JButton("Login");
        loginBtn.setBounds(110, 130, 90, 25);
        add(loginBtn);

        JButton exitBtn = new JButton("Exit");
        exitBtn.setBounds(220, 130, 90, 25);
        add(exitBtn);
```

```
        loginBtn.addActionListener(e -> doLogin());
        exitBtn.addActionListener(e -> System.exit(0));

        setLocationRelativeTo(null);
        setVisible(true);
    }

    void doLogin() {
        String u = userField.getText().trim();
        String p = new String(passField.getPassword()).trim();

        if (!users.containsKey(u)) {
            JOptionPane.showMessageDialog(this, "User not
found", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        User user = users.get(u);
        if (!user.password.equals(p)) {
            JOptionPane.showMessageDialog(this, "Wrong
password", "Error", JOptionPane.ERROR_MESSAGE);
return;
        }

        dispose();
        if (user.role == Role.DOCTOR) new DoctorFrame(user);
        else new PatientFrame(user);
    }
}

// ------------------ DOCTOR DASHBOARD (mirrors
FacultyFrame) ----------------
static class DoctorFrame extends JFrame {
    User user;
    JTable table;
    DefaultTableModel model;
```

```
    JTextField idField, nameField;
    JComboBox<String> dateBox, statusBox;
    JLabel infoLabel;

    DoctorFrame(User user) {
        this.user = user;
        setTitle("Doctor Dashboard");
        setSize(750, 520);
        setLayout(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JLabel title = new JLabel("Doctor Dashboard");
        title.setFont(new Font("SansSerif", Font.BOLD, 16));
        title.setBounds(20, 10, 300, 25);
        add(title);

        model = new DefaultTableModel(new String[]{"Appt
ID","Patient","Date","Time","Status","Notes"},0){
            public boolean isCellEditable(int r,int c){return false;}
        };

        table = new JTable(model);
        JScrollPane sp = new JScrollPane(table);
        sp.setBounds(20, 50, 700, 220);
        add(sp);

        // Add Patient (auto user creation)
        add(new JLabel("Patient ID:")).setBounds(20, 290, 80,
25);
        idField = new JTextField();
        idField.setBounds(100, 290, 120, 25);
        add(idField);

        add(new JLabel("Name:")).setBounds(240, 290, 50, 25);
        nameField = new JTextField();
        nameField.setBounds(290, 290, 140, 25);
        add(nameField);
```

```
        JButton addBtn = new JButton("Add Patient");
        addBtn.setBounds(450, 290, 120, 25);
        add(addBtn);

        // Controls
        add(new JLabel("Date (yyyy-MM-dd):")).setBounds(20,
330, 130, 25);
        dateBox = new JComboBox<>();
        dateBox.setEditable(true);
        dateBox.setBounds(150, 330, 140, 25);
        dateBox.setSelectedItem(LocalDate.now().format(DF));
        add(dateBox);

        add(new JLabel("Status:")).setBounds(310, 330, 60, 25);
        statusBox = new JComboBox<>(new
String[]{"ACCEPTED","REJECTED","PENDING"});
        statusBox.setBounds(370, 330, 100, 25);
        add(statusBox);

        JButton markBtn = new JButton("Change Status");
        markBtn.setBounds(490, 330, 140, 25);
        add(markBtn);

        JButton markAllBtn = new JButton("Mark All (for date)
Accepted");
        markAllBtn.setBounds(20, 370, 220, 25);
        add(markAllBtn);

        JButton exportBtn = new JButton("Export
Appointments");
        exportBtn.setBounds(260, 370, 180, 25);
        add(exportBtn);

        JButton chatBtn = new JButton("Chat");
        chatBtn.setBounds(460, 370, 100, 25);
        add(chatBtn);
```

```
        JButton logoutBtn = new JButton("Logout");
        logoutBtn.setBounds(620, 430, 100, 25);
        add(logoutBtn);

        infoLabel = new JLabel();
        infoLabel.setBounds(20, 410, 500, 20);
        add(infoLabel);

        // Actions
        addBtn.addActionListener(e -> addPatientAuto());
        markBtn.addActionListener(e -> changeSelectedStatus());
        markAllBtn.addActionListener(e -> markAllForDate());
        exportBtn.addActionListener(e -> exportAppointments());
        chatBtn.addActionListener(e -> openChatForSelected());
        logoutBtn.addActionListener(e -> { dispose(); new
LoginFrame(); });

        refreshTable();
        setLocationRelativeTo(null);
        setVisible(true);
    }

    void addPatientAuto() {
        String id = idField.getText().trim();
        String name = nameField.getText().trim();
        if (id.isEmpty() || name.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Enter ID &
Name", "Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }
        if (patients.containsKey(id)) {
            JOptionPane.showMessageDialog(this, "Patient ID
exists!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        patients.put(id, new Patient(id, name));
```

```
            int next = getNextPatientNumber();
            String username = "patient" + next;
            String password = String.valueOf(next) +
String.valueOf(next) + String.valueOf(next);
            users.put(username, new User(username, password,
Role.PATIENT, id));
            JOptionPane.showMessageDialog(this, "Patient
added!\nUsername: " + username + "\nPassword: " + password);
            idField.setText(""); nameField.setText("");
            refreshTable();
        }

        void refreshTable() {
            model.setRowCount(0);
            for (Appointment a : appointments.values()) {
                if (!a.doctorId.equals(user.personId)) continue;
                String pname = patients.containsKey(a.patientId) ?
patients.get(a.patientId).name : a.patientId;
                model.addRow(new Object[]{a.id, pname,
a.date.format(DF), a.time, a.status.name(), a.doctorNotes});
            }
            infoLabel.setText("Total appointments for you: " +
countMyAppointments());
        }

        int countMyAppointments() {
            int c = 0;
            for (Appointment a : appointments.values()) if
(a.doctorId.equals(user.personId)) c++;
            return c;
        }

        void changeSelectedStatus() {
            int row = table.getSelectedRow();
            if (row < 0) {
JOptionPane.showMessageDialog(this,"Select
appointment!","Warning",
JOptionPane.WARNING_MESSAGE); return; }
```

```
            String aid = (String) model.getValueAt(row,0);
            Appointment a = appointments.get(aid);
            String sel = (String) statusBox.getSelectedItem();
            try {
                a.status = Status.valueOf(sel);
            } catch (Exception ex) { a.status = Status.PENDING; }
            refreshTable();
        }

        void markAllForDate() {
            LocalDate date;
            try { date =
LocalDate.parse(dateBox.getSelectedItem().toString(), DF); }
            catch (Exception ex) {
JOptionPane.showMessageDialog(this,"Invalid
date!","Error",JOptionPane.ERROR_MESSAGE); return; }
            for (Appointment a : appointments.values()) {
                if (a.doctorId.equals(user.personId) &&
a.date.equals(date)) a.status = Status.ACCEPTED;
            }
            refreshTable();
        }

        void exportAppointments() {
            JFileChooser chooser = new JFileChooser();
            if (chooser.showSaveDialog(this) !=
JFileChooser.APPROVE_OPTION) return;
            try (PrintWriter pw = new PrintWriter(new
FileWriter(chooser.getSelectedFile()))) {
                pw.println("ID,Patient,Date,Time,Status,Notes");
                for (Appointment a : appointments.values()) {
                    if (!a.doctorId.equals(user.personId)) continue;
                    String pname = patients.containsKey(a.patientId) ?
patients.get(a.patientId).name : a.patientId;
                    pw.printf("%s,%s,%s,%s,%s,%s%n", a.id, pname,
a.date.format(DF), a.time, a.status.name(),
a.doctorNotes.replace(",", " "));
                }
```

```
            JOptionPane.showMessageDialog(this,  "Saved!");
        } catch (Exception ex) {
JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage()); }
    }

    void openChatForSelected() {
        int row = table.getSelectedRow();
        if (row < 0) {
JOptionPane.showMessageDialog(this,"Select
appointment!","Warning",
JOptionPane.WARNING_MESSAGE); return; }
        String aid = (String) model.getValueAt(row,0);
        Appointment a = appointments.get(aid);
        new ChatDialog(this, user, a).setVisible(true);
    }
}

//------------------PATIENT DASHBOARD (mirrors
StudentFrame)  -----------------
static class PatientFrame extends JFrame {
    User user;
    JLabel summaryLabel;
    JTable table;
    DefaultTableModel model;
    JComboBox<String> doctorBox, dateBox;
    JTextField timeField;

    PatientFrame(User user) {
        this.user = user;
        setTitle("Patient Dashboard");
        setSize(520, 480);
        setLayout(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JLabel title = new JLabel("Patient Dashboard");
        title.setFont(new Font("SansSerif", Font.BOLD, 16));
```

```
        title.setBounds(20, 10, 200, 25);
        add(title);

        summaryLabel = new JLabel();
        summaryLabel.setBounds(20, 40, 460, 25);
        add(summaryLabel);

        model = new DefaultTableModel(new String[]{"Appt
ID","Doctor","Date","Time","Status","Notes"},0){
            public boolean isCellEditable(int r,int c){return false;}
        };

        table = new JTable(model);
        JScrollPane sp = new JScrollPane(table);
        sp.setBounds(20, 80, 460, 220);
        add(sp);

        add(new JLabel("Choose Doctor:")).setBounds(20, 310,
100, 25);
        doctorBox = new JComboBox<>();
        doctorBox.setBounds(120, 310, 220, 25);
        add(doctorBox);
        populateDoctors();

        add(new JLabel("Date (yyyy-MM-dd):")).setBounds(20,
350, 120, 25);
        dateBox = new JComboBox<>();
        dateBox.setEditable(true);
        dateBox.setBounds(140, 350, 140, 25);

dateBox.setSelectedItem(LocalDate.now().plusDays(1).format(D
F));
        add(dateBox);

        add(new JLabel("Time:")).setBounds(300, 350, 40, 25);
        timeField = new JTextField("10:00");
        timeField.setBounds(340, 350, 100, 25);
```

```
        add(timeField);

        JButton bookBtn = new JButton("Request Appointment");
        bookBtn.setBounds(20, 390, 200, 25);
        add(bookBtn);

        JButton chatBtn = new JButton("Chat");
        chatBtn.setBounds(240, 390, 100, 25);
        add(chatBtn);

        JButton refreshBtn = new JButton("Refresh");
        refreshBtn.setBounds(360, 390, 80, 25);
        add(refreshBtn);

        JButton exportBtn = new JButton("Export My Appts");
        exportBtn.setBounds(20, 420, 200, 25);
        add(exportBtn);

        JButton logoutBtn = new JButton("Logout");
        logoutBtn.setBounds(360, 420, 120, 25);
        add(logoutBtn);

        // Actions
        bookBtn.addActionListener(e -> bookAppointment());
        chatBtn.addActionListener(e -> openChatForSelected());
        refreshBtn.addActionListener(e -> refreshData());
        exportBtn.addActionListener(e ->
exportMyAppointments());
        logoutBtn.addActionListener(e -> { dispose(); new
LoginFrame(); });

        // Welcome dialog showing accepted percentage (like
attendance %)
        updateSummaryOnLogin();

        refreshData();
        setLocationRelativeTo(null);
```

```
        setVisible(true);
    }

    void populateDoctors() {
        doctorBox.removeAllItems();
        for (Doctor d : doctors.values()) doctorBox.addItem(d.id +
" - " + d.name + " (" + d.speciality + ")");
    }

    void updateSummaryOnLogin() {
        // compute percent of patient's appointments that are
accepted (like attendance %)
        long total = 0, accepted = 0;
        for (Appointment a : appointments.values()) {
            if (!a.patientId.equals(user.personId)) continue;
            total++;
            if (a.status == Status.ACCEPTED) accepted++;
        }
        double percent = total == 0 ? 0.0 : (accepted * 100.0) /
total;
        Patient p = patients.get(user.personId);
        if (p != null) {
            JOptionPane.showMessageDialog(this,
                    "Welcome " + p.name + "\nYour Accepted
Appointment Percentage: " + String.format("%.2f", percent) +
"%");
        }
    }

    void refreshData() {
        model.setRowCount(0);
        for (Appointment a : appointments.values()) {
            if (!a.patientId.equals(user.personId)) continue;
            String dname = doctors.containsKey(a.doctorId) ?
doctors.get(a.doctorId).name : a.doctorId;
            model.addRow(new Object[]{a.id, dname,
a.date.format(DF), a.time, a.status.name(), a.doctorNotes});
```

```
        }
        // summary label
        Patient p = patients.get(user.personId);
        summaryLabel.setText(p.name + " (" + p.id + ")");
    }

    void bookAppointment() {
        String docSel = (String) doctorBox.getSelectedItem();
        if (docSel == null) {
JOptionPane.showMessageDialog(this, "Choose doctor!",
"Warning", JOptionPane.WARNING_MESSAGE); return; }
        String docId = docSel.split(" - ")[0].trim();
        String dateStr =
dateBox.getSelectedItem().toString().trim();
        String time = timeField.getText().trim();
        try {
            LocalDate date = LocalDate.parse(dateStr, DF);
            String id = nextAppointmentId();
            Appointment a = new Appointment(id, user.personId,
docId, date, time);
            appointments.put(a.id, a);
            refreshData();
            JOptionPane.showMessageDialog(this, "Appointment
requested: " + id);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Invalid
date/time", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    void openChatForSelected() {
        int row = table.getSelectedRow();
        if (row < 0) { JOptionPane.showMessageDialog(this,
"Select appointment!", "Warning",
JOptionPane.WARNING_MESSAGE); return; }
        String aid = (String) model.getValueAt(row,0);
        Appointment a = appointments.get(aid);
        new ChatDialog(this, user, a).setVisible(true);
```

```
        }

        void exportMyAppointments() {
            JFileChooser chooser = new JFileChooser();
            if (chooser.showSaveDialog(this) !=
JFileChooser.APPROVE_OPTION) return;
            try (PrintWriter pw = new PrintWriter(new
FileWriter(chooser.getSelectedFile()))) {
                pw.println("ID,Doctor,Date,Time,Status,Notes");
                for (Appointment a : appointments.values()) {
                    if (!a.patientId.equals(user.personId)) continue;
                    String dname = doctors.containsKey(a.doctorId) ?
doctors.get(a.doctorId).name : a.doctorId;
                    pw.printf("%s,%s,%s,%s,%s,%s%n", a.id, dname,
a.date.format(DF), a.time, a.status.name(),
a.doctorNotes.replace(",", " "));
                }
                JOptionPane.showMessageDialog(this,  "Saved!");
            } catch (Exception ex) {
JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage()); }
        }
    }

    // ---------- CHAT DIALOG ----------
    static class ChatDialog extends JDialog {
        Appointment appt;
        User user;
        JTextArea messageArea;
        JTextField inputField;

        ChatDialog(JFrame owner, User user, Appointment appt) {
            super(owner, "Chat - " + appt.id, true);
            this.user = user;
            this.appt = appt;

            setSize(600, 420);
```

```
        setLayout(null);

        messageArea = new JTextArea();
        messageArea.setEditable(false);
        JScrollPane sp = new JScrollPane(messageArea);
        sp.setBounds(20, 20, 540, 260);
        add(sp);

        inputField = new JTextField();
        inputField.setBounds(20, 300, 420, 30);
        add(inputField);

        JButton sendBtn = new JButton("Send");
        sendBtn.setBounds(460, 300, 100, 30);
        add(sendBtn);

        JButton closeBtn = new JButton("Close");
        closeBtn.setBounds(460, 340, 100, 30);
        add(closeBtn);

        sendBtn.addActionListener(e -> sendMessage());
        closeBtn.addActionListener(e -> dispose());

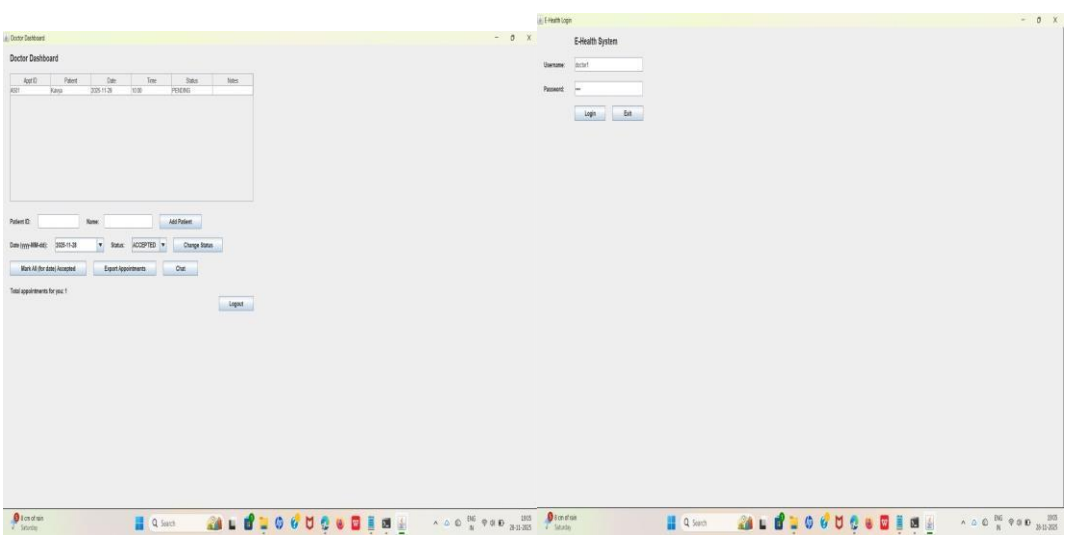
        refreshMessages();
        setLocationRelativeTo(owner);
    }

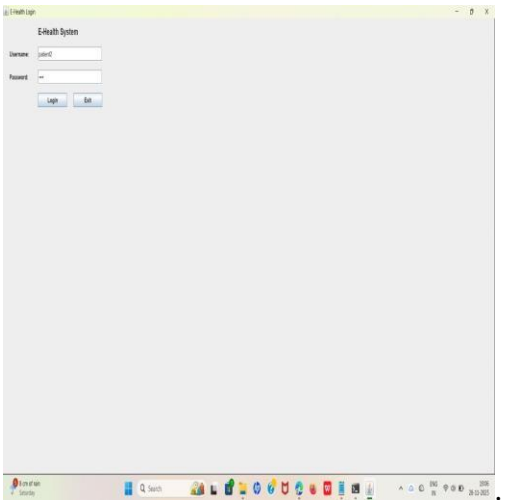
    void refreshMessages() {
        StringBuilder sb = new StringBuilder();
        for (Message m : appt.messages)
sb.append(m.toString()).append("\n");
        messageArea.setText(sb.toString());

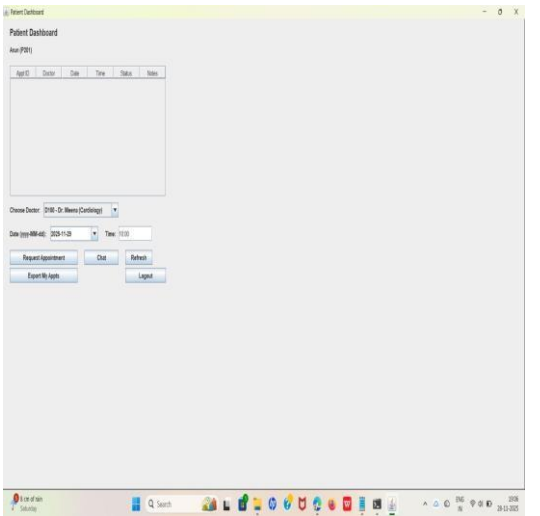
messageArea.setCaretPosition(messageArea.getDocument().getL
ength());
    }
```

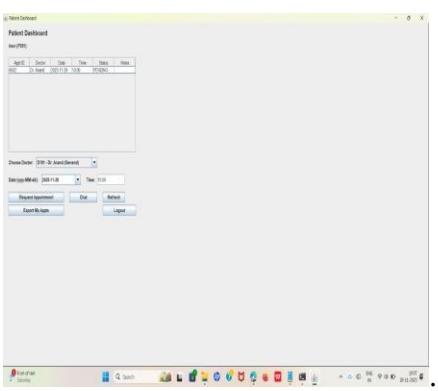
```
        void sendMessage() {
            String text = inputField.getText().trim();
            if (text.isEmpty()) return;
            Message m = new Message(user.username, user.role,
text);
            appt.messages.add(m);
            inputField.setText("");
            refreshMessages();
        }
    }
}
```

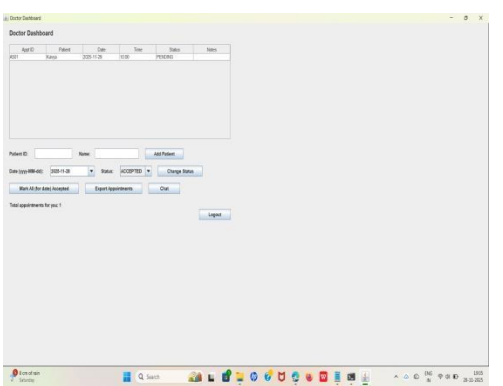
APPENDIX B(SCREENSHOTS)











REFERENCES

1. Sommerville,I.(2011).*SoftwareEngineering(9thed.)*.Boston:Addison Wesley.
2. Pressman,R.S.(2014).*SoftwareEngineering:APractitioner'sApproach(8th ed.)*. New York: McGraw-Hill.
3. Singh,R.,&Kumar,S.(2019)."AutomatedHostelManagementSystem:A Study." *International Journal of Advanced Research in Computer Science*.
4. Patel,N.,&Yadav,A.(2018)."DevelopingaHostelManagementSystem UsingWebTechnologies." *JournalofComputerScienceandTechnology*.
5. W3Schools.(n.d.)."HTML,CSS,JavaScriptTutorials."Retrieved from https://www.w3schools.com/

6. GeeksforGeeks.(n.d.)."PHPandMySQLforWebDevelopment."Retrieved from https://www.geeksforgeeks.org/
7. LaravelDocumentation.(n.d.)."OfficialLaravelDocumentation."Retrieved from https://laravel.com/docs
8. BootstrapDocumentation.(n.d.)."Bootstrap5Documentation."Retrieved fromhttps://getbootstrap.com/docs/5.0/